LAB 6

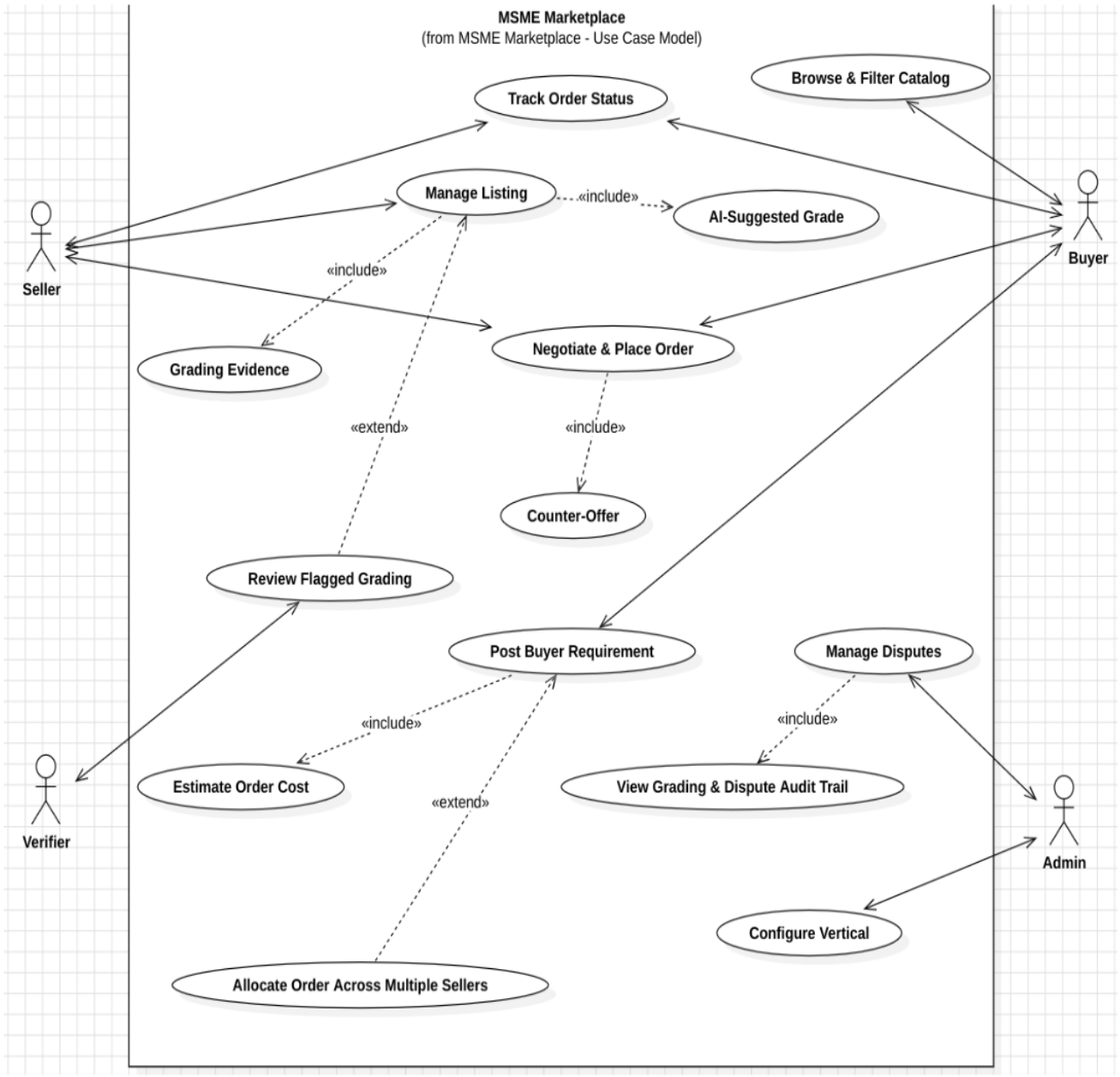
AIM – Choose three main use cases from your usecase diagram submitted for your project in Lab-4 and draw the sequence diagrams for the three use cases.

TEAM MEMBERS -

ARYAN RAJ (24BDS1094)

YASH RAJ (24BDS1209)

USE CASE DIAGRAM -



SEQUENCE DIAGRAMS (for)

1. Use Case: Manage Listings

This use case shows how a Seller will create a new commodity listing on the platform by utilizing an AI Grading Engine that evaluates the evidence provided. Based on the confidence SCORE given by AI, the listing is either accepted automatically or manually verified before publishing.

Actors:

- **Primary:** Seller
- **Secondary:** Marketplace System, AI Grading Engine, Verifier

Preconditions:

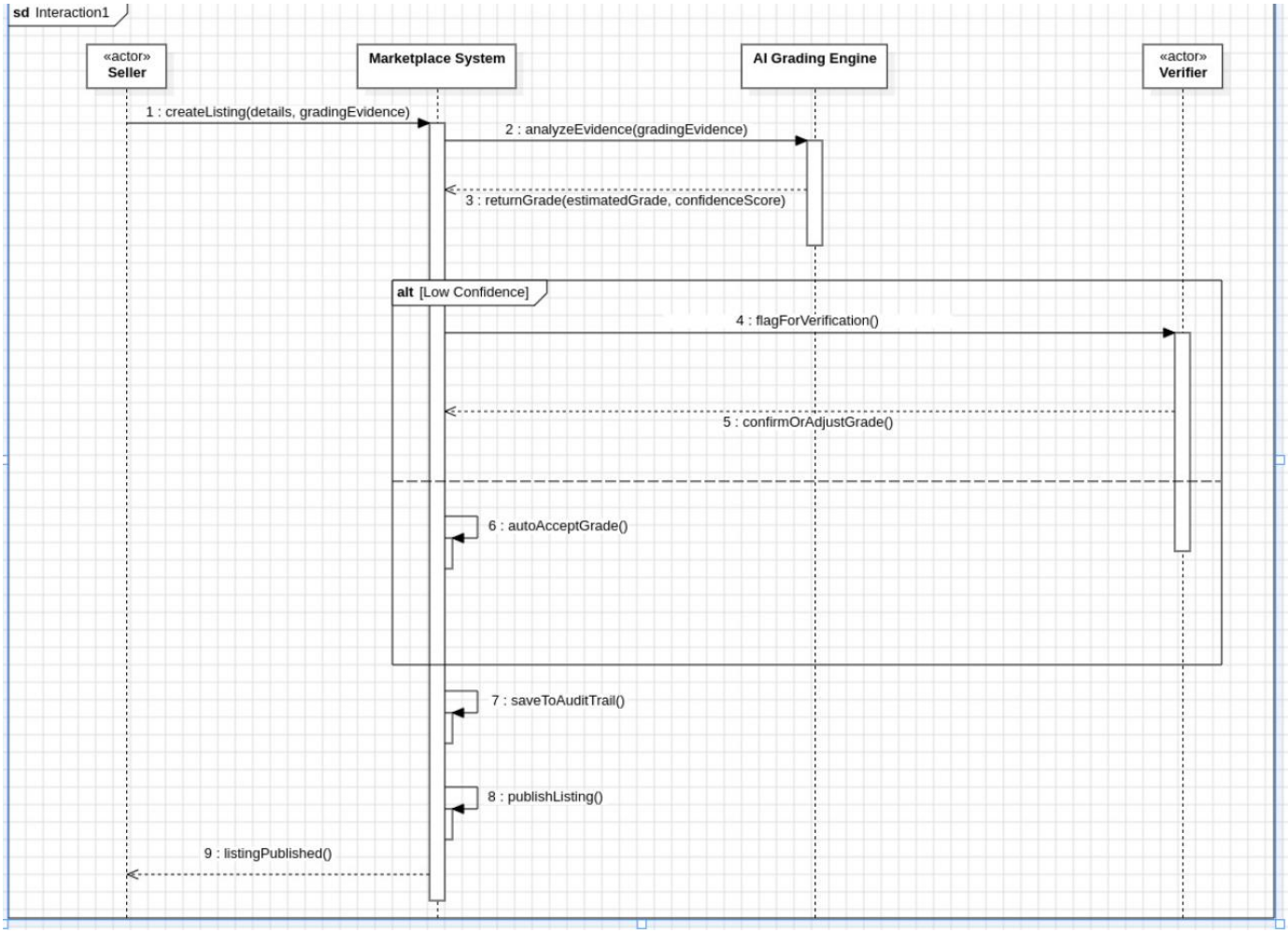
- Seller Authentication – The seller account is verified and has access to the dashboard on the marketplace.
- Required Information – The Seller has the necessary information (quantity, price, location), and proper grades (certificates/images).
- The AI Engine and Database services are running.

Basic Flow of Events (Main Success Scenario):

1. The Seller initiates the process, it submits createListing(details, gradingEvidence) to System.
2. The System sends this evidence to AI Grading Engine via analyzeEvidence().
3. The AI Engine calculates and sends back an estimated grade and a high confidence score.
4. The System views the high confidence score and executes autoAcceptGrade() to accept the grading.
5. The System records the transaction through saveToAuditTrail().
6. The System makes the listing live by using publishListing().
7. The System notifies the Seller that this process is complete (listingPublished()).

Alternative Flow (for our Low AI Confidence):

- *In Step 4:* If AI Engine returns low confidence score, system triggers alternative flow.
- The System sends a flagForVerification() request to human Verifier.
- The Verifier manually reviews all evidence and submits confirmOrAdjustGrade() response back to System.
- This flow rejoins the Basic Flow at Step 5.



2. Use Case: Negotiate and Place Orders

The use case describes how the negotiations should be done between Buyer and Seller. It provides a process where both Buyer and Seller can make counter offers until reaching an agreed price to finalize the order.

Actors:

- **Primary:** Buyer
- **Secondary:** Marketplace System, Seller

Preconditions:

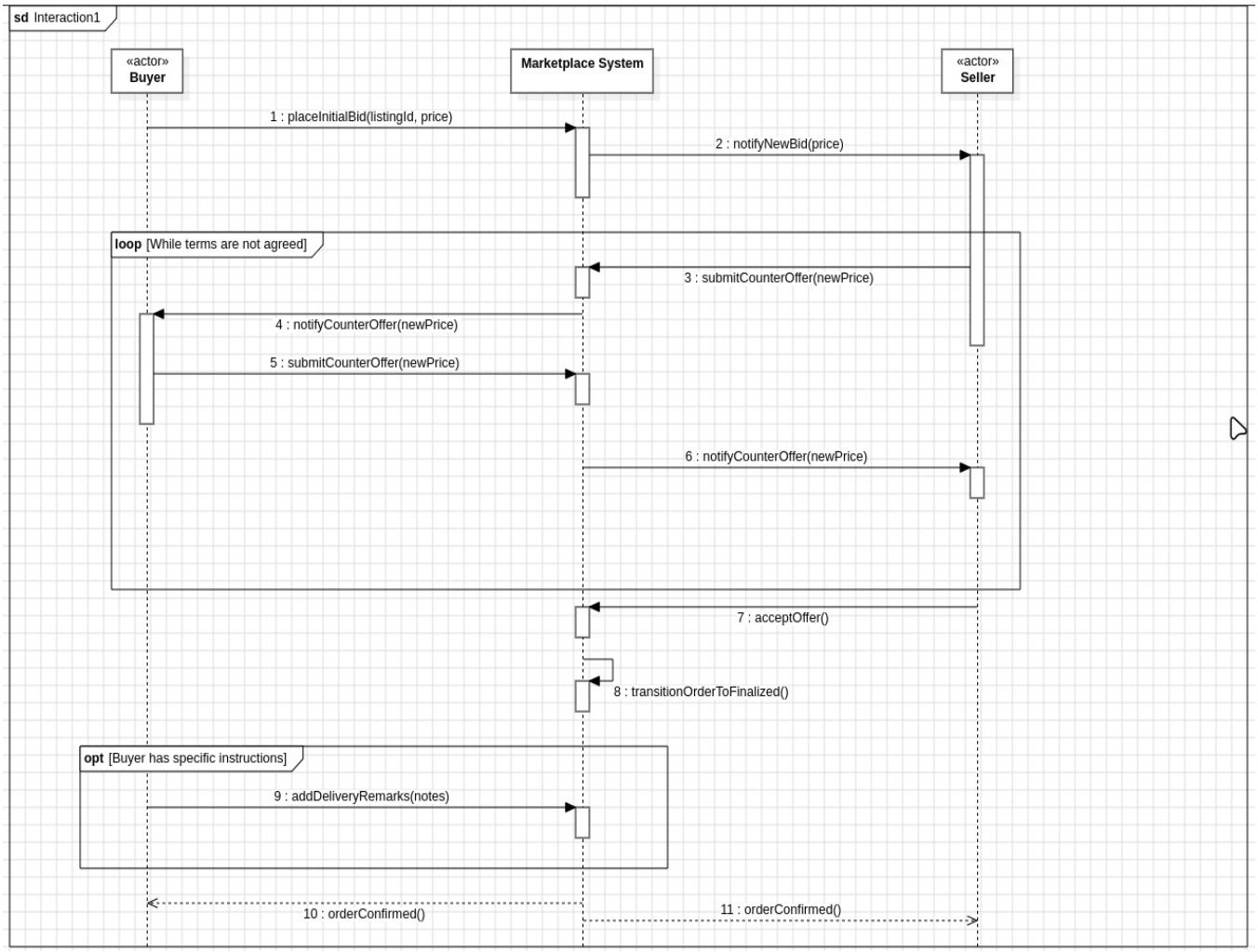
- Both the Buyer and the Seller are successfully authenticated on the platform.
- The Buyer has located an active, available listing in the catalog.
- This listing has been set up to enable bidding/negotiations.

Basic Flow of Events:

1. The Buyer will initiate negotiation by submitting placeInitialBid(listingId, price) to the System.
2. The System sends a notification to Seller with the proposed terms via notifyNewBid(price).
3. The Seller reviews the bid and accepts it immediately by sending acceptOffer() to the System.
4. The System processes the agreement by using the transitionOrderToFinalized() sequence.
5. The System sends the orderConfirmed() notification to the Buyer.

Alternative Flow (Negotiation Loop):

- *In Step 3:* If the Seller does not accept the initial bid, negotiation loop starts.
- The Seller submits submitCounterOffer(newPrice) to the System.
- The System sends notifyCounterOffer(newPrice) to the Buyer.
- The Buyer then responds with their own submitCounterOffer(newPrice).
- This cycle repeats until agreement has been reached between the two.
- Once either party accepts the offer, the loop breaks and the process rejoins the Basic Flow at Step 3.


sd Interaction1
«actor»
Buyer
Marketplace System
«actor»
Seller
1 : placeInitialBid(listingId, price)
2 : notifyNewBid(price)
loop [While terms are not agreed]
3 : submitCounterOffer(newPrice)
4 : notifyCounterOffer(newPrice)
5 : submitCounterOffer(newPrice)
6 : notifyCounterOffer(newPrice)
7 : acceptOffer()
8 : transitionOrderToFinalized()
opt [Buyer has specific instructions]
9 : addDeliveryRemarks(notes)
10 : orderConfirmed()
11 : orderConfirmed()

3. Use Case: Track Order Status

Brief Description: This use case defines the process by which a user (either a Buyer or a Seller) gets the real-time status of an order. For optimization, the system employs parallel processing to fetch internal order state data and external live shipping updates. It also includes an exception-handling mechanism to immediately abort the process if an invalid tracking request is submitted.

Actors:

- **Main:** Buyer/Seller
- **Auxiliary:** Marketplace System, Order Database, Logistics API

Preconditions:

- The user (Buyer or Seller) is logged onto the marketplace website.
- The user has a defined Order ID which can be tracked.
- The external Logistics API and internal Order Database are active and ready to respond.

Basic Flow of Events (Main Success Scenario):

1. The Buyer/Seller requests tracking information by sending the requestOrderStatus(orderId) request to the System.
2. The System verifies the orderId (skips the break fragment).
3. A concurrent process is started (par fragment):
    a. Thread A: The System calls getInternalStatus(orderId) from the Order Database. The Database processes the request and returns the returnStatus(payment, fulfillment) message.
    b. Thread B: At the same time, the System calls getLiveTracking(trackingId) from the external Logistics API. The API returns returnTracking(location, ETA).
4. After finishing with both parallel threads, the System internally calls the aggregateData() function to merge the internal status and external tracking info.
5. The System sends the displayStatus(aggregatedData) response to the Buyer/Seller.

Alternative Flow (Invalid Order ID / Break Fragment):

- In Step 2: The System validates the orderId when receiving the initial request.
- If the condition [Invalid Order ID] is fulfilled, then the system goes into the break fragment.
- The System sends a displayError("Order Not Found") response to the Buyer/Seller immediately.
- The flow ends at this step; the calls for data retrieving are not being processed.

